

K A F K A 유저 모임 밋업 / 2026.04.29

클라우드 시대의 Kafka 재설계

Evolving for Efficiency and Scalability

12년의 진화, 그리고 KIP-1150이 말해주는 것

GNU Shim · LG U+

[linkedin.com/in/gnu-shim](https://www.linkedin.com/in/gnu-shim)

오늘의 컨셉

01

과거를 짚는다

Kafka는 왜 지금처럼 설계됐는가

02

미래를 본다

KIP-1150이 어디로 가고 있는가

03

사례 한 컷

마지막 5분, 우리 팀의 AutoMQ 도입과 마이그레이션 실험

■ HOOK — 같은 워크로드, 다른 청구서

이 격차는 어디서 왔나?

Kafka + Tiered Storage

\$105K

연간 비용 (Self-hosted on AWS, 50 MiB/s Cross-AZ)

KIP-1150 Diskless Topics

\$19K

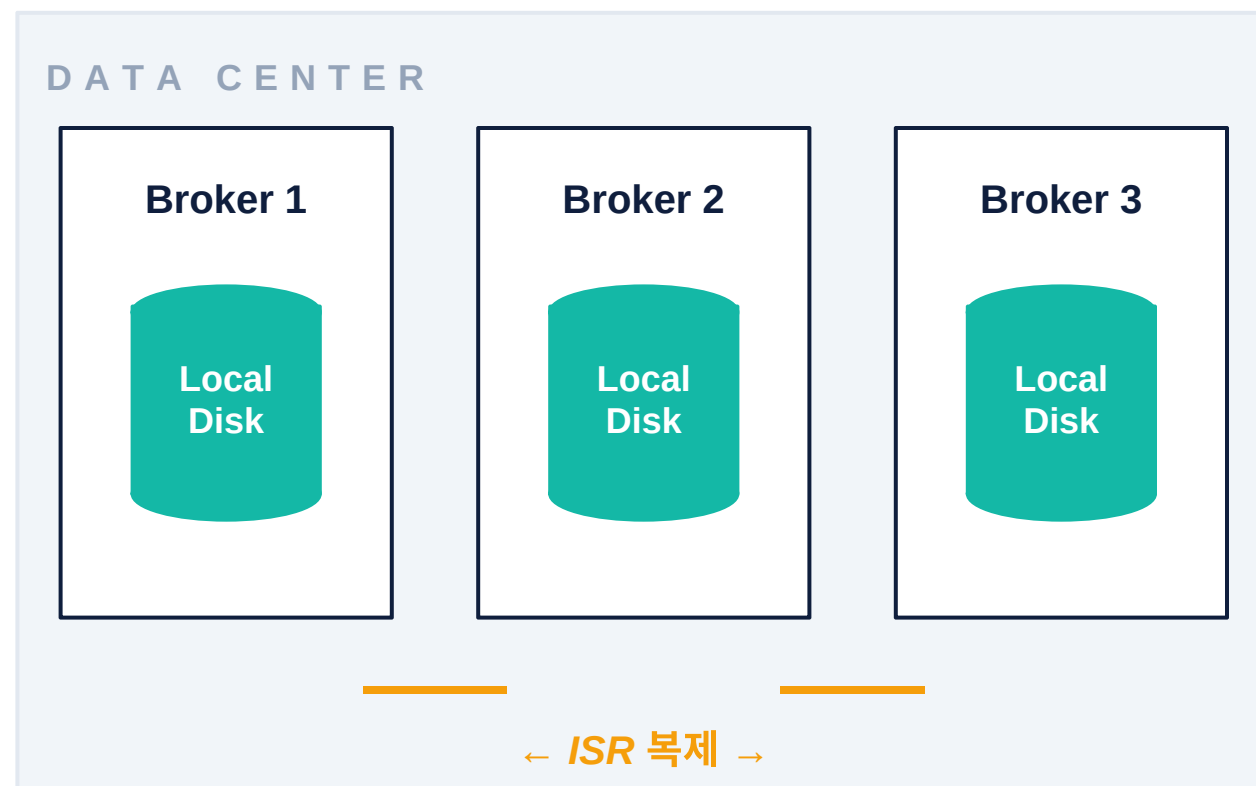
연간 비용 (동일 워크로드)

약 **5.5배** 차이. 대부분 **cross-AZ 네트워크 비용**에서 옴.

과거

Kafka는 왜 지금처럼 설계됐나

2011년, LinkedIn의 가정들



당시의 가정들

- 데이터센터 시대 — 고정된 머신, 자체 네트워크
- 네트워크는 사실상 공짜
- 디스크는 비싸고 느린 자원 — 아껴 써야 함
- Stateful 머신이 자연스러움

→ Shared Nothing + 로컬 디스크 + ISR 복제

클라우드가 깨뜨린 전제

가정 (2011)

클라우드 현실

네트워크 = 공짜



Cross-AZ는 비싸다

디스크 = 병목



S3 / NVMe 시대

Stateful = 기본



Stateless가 표준

고정 용량 계획



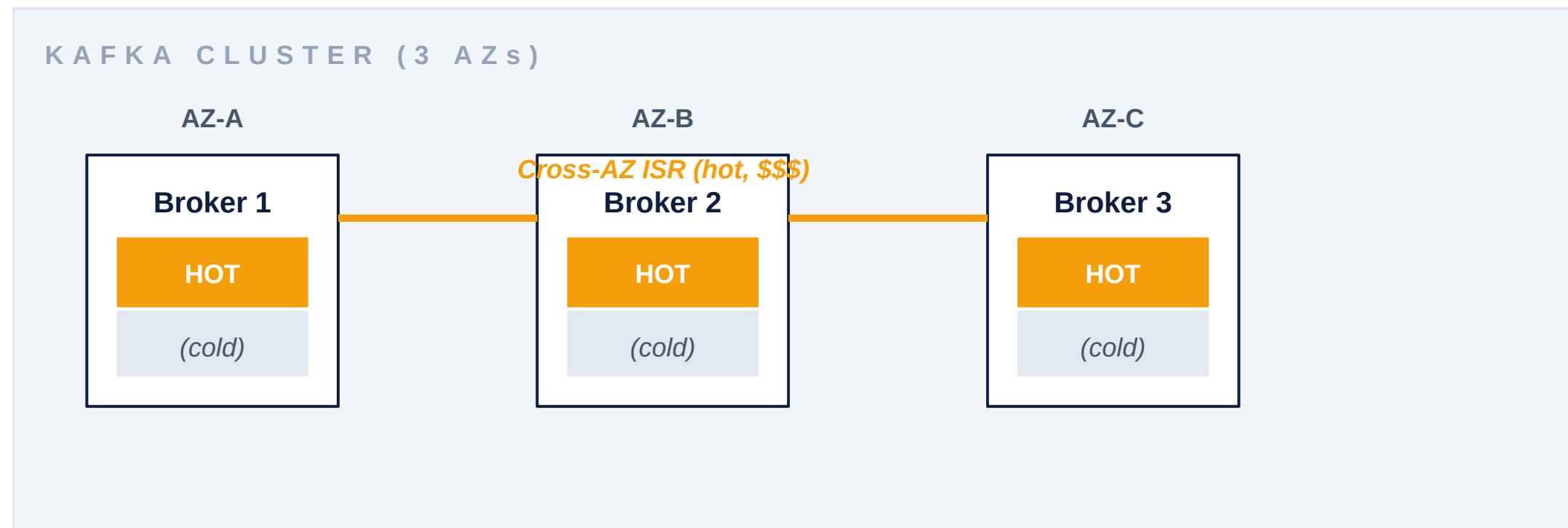
탄력 스케일링 요구

이 갭이 12년간 누적된 **기술 부채**다.

1차 시도 — Tiered Storage

KIP-405, 2024 / 절반의 해결책

콜드는 S3로, 핫은 그대로



얻은 것

Kafka 3.9 GA · 보수적 도입 · 디스크 비용 절감 · 스케일링 개선

그러나 — 절반의 해결책

활성 세그먼트는 여전히 로컬 · cross-AZ는 그대로 · 비용 본질 변화 X

본질은 **활성 경로**에 있다.

2차 시도 — 본체 밖 재설계

WarpStream · AutoMQ · Confluent Freight

같은 시기, 세 솔루션

2023

WarpStream

- 프로토콜 호환
- 클로즈드 소스
- Confluent 인수 (2024)

2023

AutoMQ

- 오픈소스 (Apache 2.0)
- EBS WAL (Enterprise only) + S3
- 프로토콜 100% 호환

2025 Q1

Confluent Freight

- 완전 매니지드
- S3 직접 쓰기
- Confluent Cloud 한정

같은 답에 수렴 — *Stateless* 브로커 + S3 단일 내구성 + *Cross-AZ* 복제 제거

같은 답, 갈라진 길

공통 합의

- Stateless 브로커
- S3 단일 내구성
- Cross-AZ 복제 제거
- 수 초 레이턴시 수용

갈라진 노선

라이선스	오픈소스 vs 상용
호환성	프로토콜 호환 vs 자체 구현
운영 모델	셀프호스팅 vs 매니지드
스토리지 백엔드	S3 직접 vs EBS+S3

인사이드

Kafka는 바깥에서 재설계되고 있었다

그리고 본체는 이걸 보고 있었다.

수 렴

3개의 KIP가 동시에 등장한 사건

2025년 4월 — 한꺼번에 셋

K A F K A 역사상 전례 없는 일

KIP 1150 Aiven 혁명 Leaderless 재설계	KIP 1176 Slack 점진 Tiered 확장	KIP 1183 AutoMQ 추상화 스토리지 레이어
---	--	---

같은 문제를 셋이 동시에 풀려고 한 건 — 우연이 아니라 필연

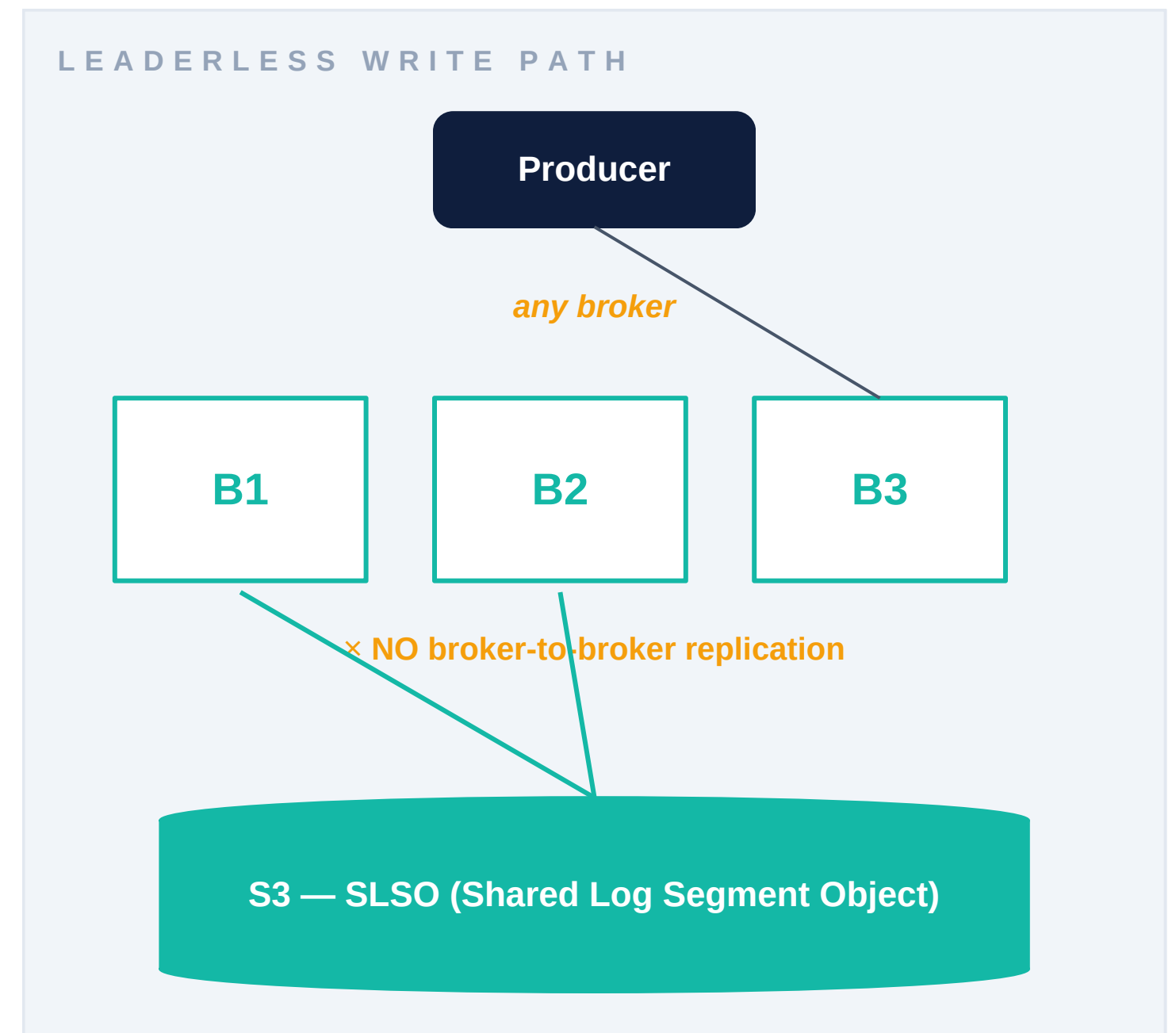
KIP-1150 — Aiven의 혁명

REVOLUTION

리더-팔로워 모델 자체를 폐기

- 어떤 브로커든 produce/fetch
- 여러 파티션을 하나의 S3 오브젝트에 (SLSO)
- Batch Coordinator가 오프셋 시퀀싱
- 프로듀서가 공유 WAL에 직접 기록

가장 과감한 재설계.



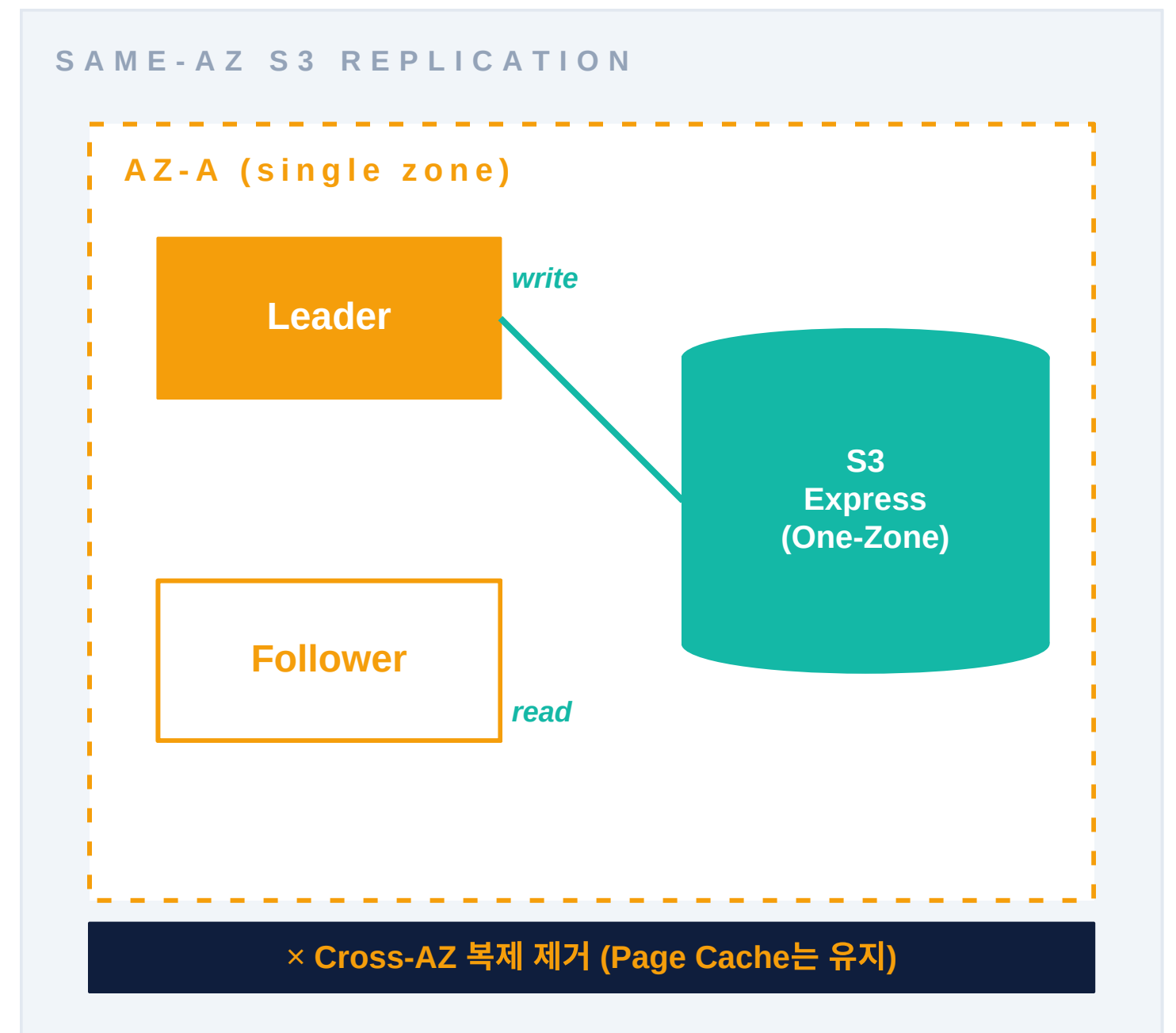
KIP-1176 — Slack의 점진

EVOLUTION

기존 Kafka의 "배관"만 최적화

- 활성 세그먼트도 S3 업로드 (KIP-405 확장)
- 팔로워는 같은 AZ S3에서 복제
- Page Cache 유지 — 밀리초 레이턴시
- Cross-AZ 트래픽 약 43% 절감

기존 자산을 가장 많이 살리는 보수적 접근.



KIP-1183 — AutoMQ의 추상화

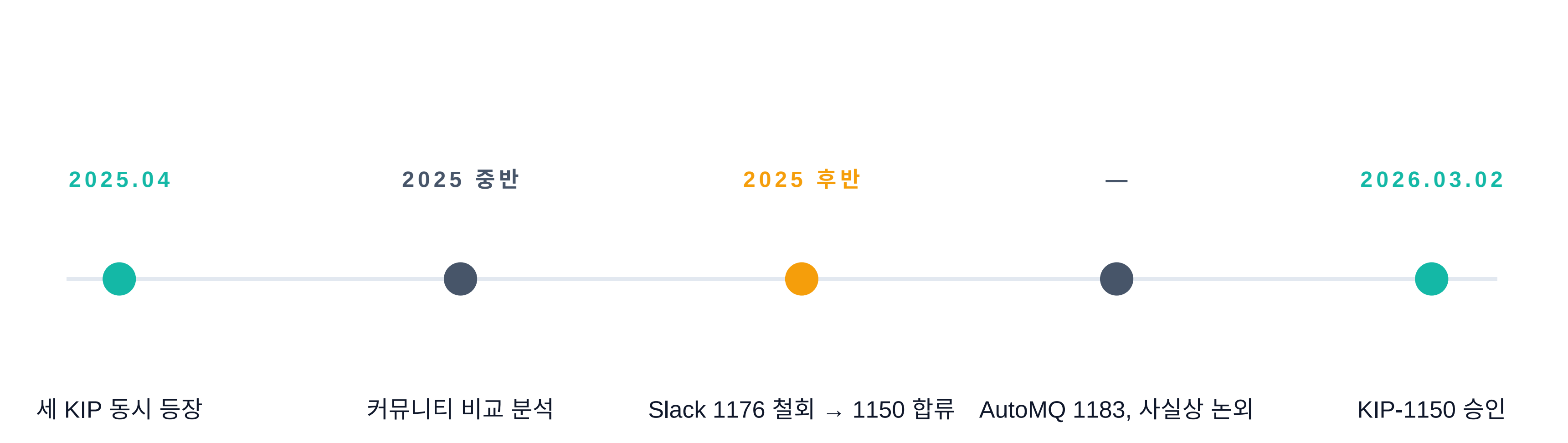
ABSTRACTION

스토리지 엔진 플러그인 구조

- AbstractLog / AbstractLogSegment — 베이스 타입 도입
- 기존 UnifiedLog는 파일 기반 구현으로 유지
- S3/HDFS/NFS용 SharedLog 구현 플러그인 가능
- 리더 기반 유지, RF=1, 내구성은 공유 스토어에 위임

AutoMQ 자체 구현체를 커뮤니티에 기여하겠다는 취지.

10개월의 논의



최종 표결 Binding 9 · Non-binding 5 — 압도적 합의

최종 KIP-1150은 무엇인가

Aiven 뼈대

Leaderless write path · 직접 S3 기록

혁명적 설계 채택



Slack 경로

기존 Tiered Storage 재활용

점진적 경로 흡수



하이브리드

KIP-1150

(2026.03 승인)

INSIGHT

커뮤니티는 혁명을 택했다 · 대신 점진적 경로를 흡수했다 · 단일 벤더의 추상화는 받지 않았다

현재

KIP-1150이 실제로 무엇인가

핵심 개념 셋

01

Leaderless

어떤 브로커든 produce/fetch

02

SLSO

여러 파티션을 하나의 S3 오브젝트에

03

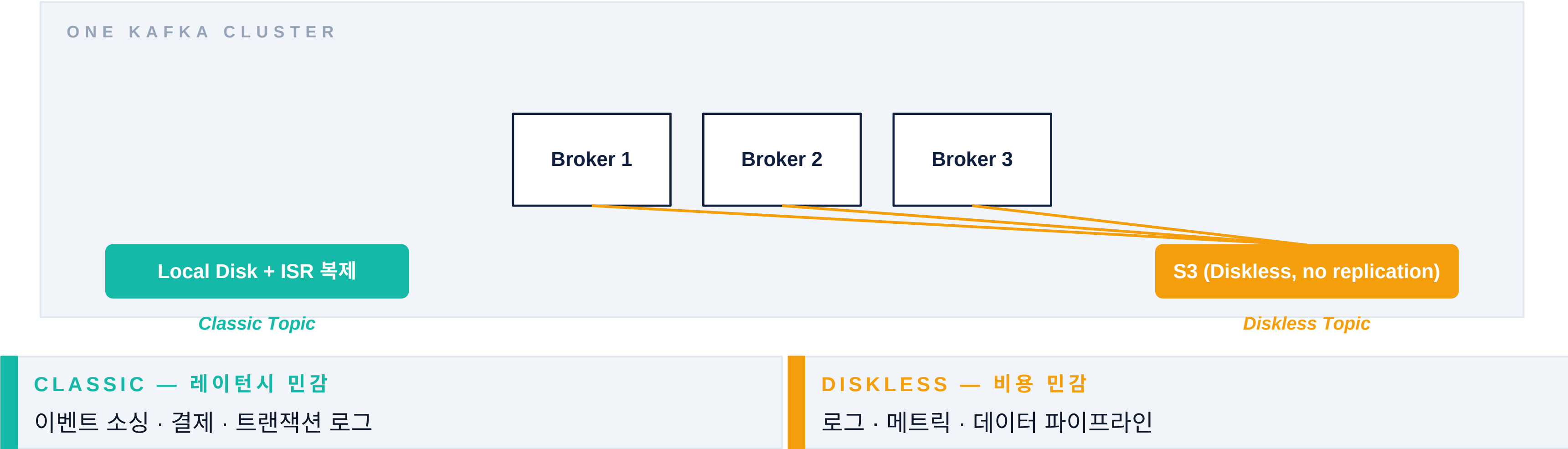
Batch Coordinator

오프셋 시퀀서 컴포넌트

(이 자리에서 깊게 들어가지는 않습니다)

진짜 핵심

Classic ↔ Diskless, 한 클러스터 혼용



별도 클러스터 운영 X — 운영 표면적이 그대로

트레이드오프는 분명하다

P99 LATENCY

3 — 3.5초

Aiven 내부 측정 기준

기본 P99	3 ~ 3.5 초
튜닝 시 하한	약 2 초
서브 100ms	불가능

모든 토픽이 갈 수 있는 게 아니다
워크로드가 갈라진다

미래

그래서 우리는 어디로 가나

셋만 짚자

EFFICIENCY

Cross-AZ 비용 구조에서 해방됐다

12년 만에 클라우드 비용 모델과 정렬

SCALABILITY

Stateless 브로커로 초 단위 스케일링

Stateful 시대의 "시간~일"이 "초"가 됐다

그 래 서 다 음 은 ?

워크로드별 토픽 분리가 표준이 된다

다음 부채 후보 — *KRaft* 메타데이터 · 클라이언트 프로토콜

12년이 한 일

Shared Nothing → Stateless

로컬 디스크 → 오브젝트 스토리지

ISR 복제 → S3 단일 내구성

데이터센터 가정 → 클라우드 가정

12년 만에 *Kafka*가 클라우드를 따라잡았다.

다음 12년은?

우리는 어디까지 와 있나

AutoMQ 도입과 마이그레이션 실험

로그는 갔고, 이벤트 소싱은 멈췄다

LOG PIPELINE — 도입

로그 파이프라인

→ AutoMQ

- 수 초 레이턴시 허용
- Cross-AZ 비용이 가장 큰 워크로드
- 비용 민감 / 레이턴시 둔감의 정석

EVENT SOURCING — 보류

이벤트 소싱

→ Classic 유지

- 서브-100ms 레이턴시 요구
- 강한 순서 보장 / 트랜잭션 의미론
- 운영 검증 부족

본편 결론과 일치 — "워크로드가 갈라진다"의 살아있는 증거

운영 중인 데이터를 어떻게 옮기나?

표준 답

MirrorMaker2

한계

- 원본 클러스터에 부하 (Consumer 경로)
- Consumer throughput에 묶이는 속도
- 라이브 운영 필수

다른 길은? — 로컬 .log 파일을 직접 읽으면?

→ 이 질문에서 출발한 실험: github.com/gnu-gnu/bangbang

bangbang

정방향 — AutoMQ → Kafka

S3 compacted object + WAL → .log 파일

byte-for-byte 무손실 (RecordBatch raw bytes 보존)

역방향 — Kafka → AutoMQ

.log 파일 → Producer API → AutoMQ

offset 재채번 → 매핑 테이블로 추적 (TSV)

모듈 구조

bangbang-ir	중간 표현 (PartitionData, RawRecordBatch)
bangbang-kafka	Kafka log segment Reader / Writer
bangbang-s3stream	S3Stream Reader · Producer Importer · KRaft 파서
bangbang-cli	CLI (kafka2automq · automq2kafka · inspect)

MM2 vs bangbang

	MirrorMaker2	bangbang
데이터 소스	라이브 클러스터	로컬 .log 파일
원본 부하	Consumer 부하 발생	없음 (파일 직접 읽기)
속도	Consumer throughput	디스크 I/O 속도
Offset 보존	재채번 (매핑 제공)	재채번 (매핑 제공)
동작 방식	온라인만	오프라인 가능
설정 복잡도	Connector 관리	CLI 한 줄

표준 도구가 닿지 않는 영역을 직접 파봐야 알 수 있는 것들이 있다

알아낸 것 · 그리고 한계

알아낸 것

- KRaft __cluster_metadata 파싱 → stream ↔ 파티션 매핑 가능
- Kafka / AutoMQ S3Stream 바이너리 포맷의 호환 영역과 비호환 영역 분리
- WAL Builder ↔ Parser byte-for-byte 라운드트립 통과

그리고 한계

- AutoMQ Controller의 epoch 검증 → 외부에서 KRaft 직접 commit 불가
- 결국 Producer 경로 우회 필요
- "진짜 무손실 역방향"은 현재 막혀 있음
- 대용량 처리 시 메모리 로드 — 스트리밍 개선 필요

다음 12년의 *Kafka*는 — 이런 실험들로 채워질 것

2.2B 메시지/일 via AutoMQ

2.2B

일일 메시지 처리량

AWS ECS 경유

100%

Stateless 브로커

로컬 스토리지 의존성 0

100%

Observability 호환

Fluentd · Sumo Logic · OpenSearch

아키텍처 핵심

- AWS ECS (Private VPC, 내부 DNS)
- 하이브리드 클라우드 (Public AWS + Private)
- Terraform IaC 표준화
- ECS Circuit Breaker · Rolling Update (zero downtime)
- S3 기반 로그 retention (최대 3일)
- 기존 toolchain 무수정 (Fluentd · Sumo · OpenSearch)

Q & A

질문 / 코멘트 / 반박 환영

감사합니다

Evolving for Efficiency and Scalability